ADGSTUDIOS

DisputeGuard

Software

Requirements Specification

Version 1.0

System:	ADGSTUDIOS DisputeGuard
Document:	Software Requirements Specification (SRS)
Status:	Baseline for implementation planning
Date:	October 2026

A testable description of the system's externally observable behaviour, constraints and acceptance conditions.

Contents

1 Introduction **2**
- 1.1 Purpose . . . 2
- 1.2 Conventions . . . 2
- 1.3 System Scope . . . 2
- 1.4 Terms . . . 2

2 Overall Description **3**
- 2.1 Product Perspective . . . 3
- 2.2 User Classes . . . 3
- 2.3 Operating Environment . . . 3
- 2.4 Design and Implementation Constraints . . . 3
- 2.5 Assumptions and Dependencies . . . 3

3 External Interface Requirements **4**
- 3.1 User Interfaces . . . 4
- 3.2 API Interfaces . . . 4
- 3.3 Service Interfaces . . . 4

4 Functional Requirements **5**
- 4.1 Authentication and Access Control . . . 5
- 4.2 Transaction Retrieval and Eligibility . . . 5
- 4.3 Dispute Creation and Lifecycle . . . 5
- 4.4 Idempotency and Concurrent Updates . . . 6
- 4.5 Evidence and Analyst Notes . . . 6
- 4.6 Risk Indicators . . . 6
- 4.7 AI-Assisted Analysis . . . 7
- 4.8 Audit, Operations and Background Work . . . 7

5 Data Requirements **8**

6 Non-Functional Requirements **8**
- 6.1 Security and Privacy . . . 8
- 6.2 Reliability and Integrity . . . 8
- 6.3 Performance and Scalability . . . 9
- 6.4 Maintainability and Portability . . . 9
- 6.5 Usability and Accessibility . . . 9

7 Verification and Acceptance **9**

8 Traceability **10**

9 Open Decisions **10**

1 Introduction

1.1 Purpose

This Software Requirements Specification defines the required behaviour, external interfaces, quality attributes and acceptance conditions for ADGSTUDIOS DisputeGuard. It is intended for product stakeholders, implementers, testers and reviewers.

This document is a requirements baseline, not a claim that the system has already been implemented, independently certified, or approved for use in a live financial institution. Production deployment requires separate operational, legal, regulatory and business approval.

1.2 Conventions

Each normative requirement has a unique identifier. The term **shall** denotes a mandatory requirement. **Should** denotes a recommendation rather than an acceptance condition. Each requirement includes a verification method: **Test**, **Inspection**, or **Analysis**.

Requirements are phrased to be independently verifiable. Values that the current product definition does not establish are marked **TBD**; implementation must not silently treat an example value as approved business policy.

1.3 System Scope

DisputeGuard is a demonstration-oriented card transaction dispute management system. It supports a customer-facing workflow for viewing mock transactions, creating disputes and following their progress, and an analyst workflow for reviewing cases, evidence and deterministic signals.

PostgreSQL is the authoritative store for case data. Redis may support caching, rate limiting and asynchronous coordination but is not the source of truth. Optional local AI assistance may classify, summarise and recommend investigation steps. An authorised human retains responsibility for final case decisions.

The current scope does not include direct card-network connectivity, real settlement or money movement, a production identity provider, or an approved regulatory process unless those are separately specified.

1.4 Terms

Term	Meaning
Customer	Authenticated user who may access their own transactions and disputes.
Analyst	Authorised user who reviews disputes and performs permitted case actions.
Administrator	Authorised user who manages approved system configuration and access.
Dispute	A case raised about a transaction and processed through a defined lifecycle.
Evidence	A customer-provided file or other supporting material attached to a dispute.
Active dispute	A dispute whose state belongs to the configured active-state set.
Risk signal	A deterministic indicator intended to support analyst triage, not decide a case.
AI analysis	Optional model-generated assistance subject to application validation and limits.
TBD	A product or operational decision that remains to be approved.

2 Overall Description

2.1 Product Perspective

The system consists of customer and analyst interfaces, a backend API, a domain and business-rule layer, PostgreSQL persistence, and optional Redis and AI worker services. A local Docker-based deployment is the baseline evaluation environment. Kubernetes is an optional deployment model and is not required for local evaluation.

2.2 User Classes

- **Customer:** views their own mock transactions, creates disputes for eligible transactions, submits supporting material and checks dispute progress.
- **Dispute analyst:** reviews disputes, examines transaction information and risk indicators, requests evidence, records notes and performs permitted state transitions.
- **Administrator:** manages explicitly authorised users and administrative configuration.
- **Operator:** configures and monitors the deployment, services, logs, metrics and background workers.
- **AI worker:** optional service identity that performs bounded, allowlisted analysis; it is not a human user or decision authority.

2.3 Operating Environment

The intended application stack is a web frontend, a Rust HTTP API, PostgreSQL and optional Redis and Ollama services. Docker provides a reproducible local runtime. Specific supported browser versions, operating-system versions, resource minima and production hosting targets are TBD and require product approval.

2.4 Design and Implementation Constraints

- PostgreSQL is authoritative for persisted transactions, disputes, events and audit records.
- Redis data shall be treated as ephemeral or rebuildable.
- Authoritative monetary values shall use integer minor units or an exact fixed-precision decimal representation, not binary floating point.
- Server-side domain rules, authentication and authorisation remain authoritative regardless of client-side behaviour.
- AI assistance shall not approve or reject disputes, issue refunds, alter balances or bypass access control.
- Kubernetes is optional for the baseline deliverable.

2.5 Assumptions and Dependencies

The baseline uses seeded or otherwise mock transaction data. The database, object storage, malware scanner, identity provider and model provider may be external dependencies in a future deployment; their availability and service-level commitments are not established by this SRS. AI functionality is optional and its absence shall not prevent the core dispute workflow from operating.

3 External Interface Requirements

3.1 User Interfaces

SRS-UI-001 **(Inspection and end-to-end test)** The customer interface shall provide authenticated views of the customer's transactions, dispute submission, evidence submission and dispute status history.

SRS-UI-002 **(Inspection and role-based end-to-end test)** The analyst interface shall provide an authorised case queue, case details, evidence access, risk indicators, notes, permitted actions and audit history.

SRS-UI-003 **(Inspection)** The interface shall identify AI-generated content as assistance and shall not present it as a final case decision.

SRS-UI-004 **(Negative-path end-to-end test)** The interface shall display validation, authorisation, conflict and service errors without implying that a failed operation succeeded.

SRS-UI-005 **(End-to-end test)** The customer interface shall display a stable dispute reference and the current persisted status after successful dispute creation.

3.2 API Interfaces

The following route families define the intended interface surface; the exact request schemas, response schemas, pagination convention and error format shall be documented in the API contract before release.

- Authentication: `/api/v1/auth/*`
- Customer transactions: `/api/v1/transactions`
- Customer disputes: `/api/v1/disputes`
- Dispute evidence: `/api/v1/disputes/:id/evidence`
- Dispute timeline: `/api/v1/disputes/:id/timeline`
- Analyst cases and actions: `/api/v1/analyst/disputes/*`
- Health: `/health` and `/ready`

SRS-API-001 **(API integration test)** The API shall expose authenticated operations for customer transaction retrieval, dispute creation and retrieval, and dispute timeline retrieval.

SRS-API-002 **(API integration test)** The API shall expose separately authorised analyst operations for case retrieval, state transitions, notes and optional AI analysis.

SRS-API-003 **(API negative tests and contract inspection)** The API shall validate request data at the server boundary and return a documented error for invalid input.

SRS-API-004 **(Deployment integration test)** The API shall provide health and readiness endpoints that distinguish process health from readiness to serve application traffic.

SRS-API-005 **(Inspection)** The API contract shall specify authentication, authorisation, request and response schemas, error responses, pagination and upload limits before release.

3.3 Service Interfaces

SRS-INT-001 **(Architecture inspection and integration test)** The backend shall use PostgreSQL as the authoritative persistence interface for dispute state and related durable records.

SRS-INT-002 **(Failure-injection test)** The application shall remain usable for core dispute operations when Redis is unavailable, subject to the explicitly configured rate-limit and background-job degradation policy.

SRS-INT-003 **(Integration test)** If AI inference is enabled, the application shall communicate with the configured model service through bounded application-controlled requests.

SRS-INT-004 **(Integration test and inspection)** Background workers shall communicate job outcomes and failures through durable application records or operational telemetry.

4 Functional Requirements

4.1 Authentication and Access Control

SRS-FUN-001 **(API authorisation test)** The system shall require an authenticated identity before providing non-public transaction or dispute data.

SRS-FUN-002 **(Role-based API test)** The system shall enforce role permissions on the server for customer, analyst and administrator operations.

SRS-FUN-003 **(Cross-customer access test)** The system shall determine customer transaction ownership using the authenticated identity and persisted ownership data, not a client-supplied identifier alone.

SRS-FUN-004 **(IDOR negative test)** A customer shall not retrieve another customer's transaction, dispute or evidence by changing an identifier in a request.

SRS-FUN-005 **(Role-based API test)** The system shall reject an analyst-only operation when the authenticated actor lacks the required permission.

SRS-FUN-006 **(Inspection and authorisation tests)** Administrative permissions shall be explicitly assigned and checked server-side; they shall not be inferred from user-interface visibility or analyst status.

4.2 Transaction Retrieval and Eligibility

SRS-FUN-010 **(API and end-to-end test)** The system shall present a customer only transactions that the authenticated customer is authorised to view.

SRS-FUN-011 **(Dispute creation integration test)** Before creating a dispute, the system shall verify transaction ownership and the configured eligibility policy on the server.

SRS-FUN-012 **(Inspection)** The transaction eligibility rules, including any time window, transaction status, amount, reason or exception criteria, shall be documented configuration before release.

SRS-FUN-013 **(Negative API test)** When a transaction is not eligible, the system shall reject dispute creation and return an explicit, safe explanation that does not disclose another user's information.

4.3 Dispute Creation and Lifecycle

SRS-FUN-020 **(Database integration test)** The system shall persist a successfully created dispute with a unique, stable reference and an initial `SUBMITTED` state.

SRS-FUN-021 **(State-machine test)** The system shall maintain dispute states from the configured lifecycle, including `SUBMITTED`, `UNDER_REVIEW`, `EVIDENCE_REQUIRED`, `EVIDENCE_RECEIVED`, `INVESTIGATING`, `APPROVED`, `REJECTED`, `REFUND_PENDING` and `RESOLVED`, subject to the approved transition table.

SRS-FUN-022 **(State-machine negative test)** The system shall reject any state transition not present in the configured transition table.

SRS-FUN-023 **(Authorisation and state-machine tests)** The system shall verify the actor's permission for each requested transition independently of whether that transition is valid for the current state.

SRS-FUN-024 **(Database integration test)** The system shall persist each successful state change with its previous state, new state, actor, event and timestamp.

SRS-FUN-025 **(Role-based end-to-end test)** The system shall expose the persisted dispute history to its authorised customer and analyst views.

SRS-FUN-026 **(Concurrent database integration test)** The system shall enforce the approved set of active states and the rule that no transaction has more than one active dispute.

SRS-FUN-027 **(Database failure-injection test)** Dispute creation, its initial event, its audit record and its idempotency result shall commit atomically or be rolled back together.

4.4 Idempotency and Concurrent Updates

SRS-FUN-030 **(API integration test)** The dispute-creation API shall support an idempotency key scoped to the authenticated user.

SRS-FUN-031 **(Idempotency integration test)** A retry using the same user, key and canonical request shall return the original result without creating another dispute.

SRS-FUN-032 **(Idempotency negative test)** Reuse of an idempotency key with a different canonical request shall be rejected without creating a new dispute.

SRS-FUN-033 **(Concurrency integration test)** The system shall compare a caller's expected dispute version with the persisted version before applying a versioned update.

SRS-FUN-034 **(Concurrent-update test)** The system shall reject a stale update and shall not overwrite a newer persisted dispute state.

4.5 Evidence and Analyst Notes

SRS-FUN-040 **(Evidence access-control test)** The system shall authorise evidence upload and retrieval against the associated dispute and authenticated actor.

SRS-FUN-041 **(Upload boundary tests)** The system shall enforce configured maximum upload size, allowed file types and server-side filename generation.

SRS-FUN-042 **(Malformed and mismatched upload tests)** The system shall validate uploaded content independently of the filename extension and client-declared MIME type.

SRS-FUN-043 **(Deployment inspection and integration test)** The system shall store uploaded evidence outside directly executable web paths and persist its association and metadata.

SRS-FUN-044 **(Scanner failure-injection test)** When malware scanning is required, the system shall not mark evidence clean unless an available scanner reports a clean result.

SRS-FUN-045 **(Audit integration test)** The system shall record an audit event for evidence submission and authorised evidence-related actions.

SRS-FUN-046 **(Role-based integration test)** The analyst workflow shall permit authorised notes to be associated with the relevant dispute and shall preserve their actor and creation time.

SRS-FUN-047 **(Inspection)** Evidence retention, deletion, encryption and malware-scan policy shall be approved and documented before production use.

4.6 Risk Indicators

SRS-FUN-050 **(Unit and end-to-end tests)** The system shall calculate and present deterministic risk signals separately from final dispute decisions.

SRS-FUN-051 **(Unit test and configuration validation)** The system shall use configured, non-negative component weights whose sum is one when calculating a weighted risk score.

SRS-FUN-052 **(Unit test)** The system shall not substitute a missing or mathematically undefined signal with a zero-risk value unless an approved missing-data policy explicitly requires that treatment.

SRS-FUN-053 **(Inspection)** The system shall identify risk scores and anomaly indicators as advisory and not as proof of fraud.

SRS-FUN-054 **(Configuration inspection)** Risk weights, normalisation parameters, thresholds, reference populations and currency handling shall be documented configuration before evaluation.

SRS-FUN-055 **(Property-based test)** The system shall avoid presenting a risk score outside the defined range $[0, 1]$ when its configured inputs and weights meet the documented preconditions.

4.7 AI-Assisted Analysis

SRS-FUN-060 **(Configuration inspection and integration test)** When enabled, AI analysis shall be limited to configured assistance such as classification, summarisation and evidence recommendations.

SRS-FUN-061 **(Schema-validation tests)** The system shall validate AI output against an explicit schema before storing or presenting it as structured analysis.

SRS-FUN-062 **(Architecture inspection and negative integration test)** The AI worker shall use only application-allowlisted tools and shall not receive unrestricted SQL execution capability.

SRS-FUN-063 **(Budget-exhaustion tests)** The system shall enforce configured iteration, tool-call, token and elapsed-time limits for bounded agent executions.

SRS-FUN-064 **(Prompt-injection test and inspection)** Customer descriptions and uploaded evidence shall be treated as untrusted data and shall not grant an AI worker additional authority.

SRS-FUN-065 **(Negative integration test)** AI analysis shall not approve or reject a dispute, issue a refund, alter a balance, modify access permissions or bypass a human-authorised workflow.

SRS-FUN-066 **(Failure-injection test)** If AI inference or a worker is unavailable, the core dispute workflow shall remain available and failed or incomplete AI work shall be identified as such.

SRS-FUN-067 **(Log inspection and integration test)** The system shall record structured AI execution events and tool outcomes without exposing private model chain-of-thought.

SRS-FUN-068 **(Database integration test and inspection)** The system shall associate AI analysis with its case, model/configuration metadata and completion or failure status, subject to approved data-retention policy.

4.8 Audit, Operations and Background Work

SRS-FUN-070 **(Audit coverage test)** The system shall create an audit event for each security-sensitive operation and case lifecycle mutation.

SRS-FUN-071 **(Authorisation and persistence tests)** Ordinary application users shall not be able to silently rewrite or delete historical audit events.

SRS-FUN-072 **(Log inspection and integration test)** The system shall emit structured request logs containing a correlation identifier and safe error classification.

SRS-FUN-073 **(Log review and automated redaction test)** The system shall not include credentials, secrets or unnecessary sensitive customer data in routine logs.

SRS-FUN-074 **(Metrics endpoint test)** The system shall expose operational measurements for request rate, error rate, latency percentiles, active disputes, queue depth, AI job duration and database-pool utilisation where the corresponding component is enabled.

SRS-FUN-075 **(Worker retry tests)** A background operation shall be retried only when it is explicitly classified as retryable and protected against duplicate effects.

SRS-FUN-076 **(Health-check integration test)** The system shall report readiness separately from process liveness and shall not report readiness while required application dependencies are unavailable.

5 Data Requirements

SRS-DAT-001 **(Schema inspection and integration tests)** The system shall persist users, transaction references, disputes, dispute events, evidence metadata, analyst notes, audit events and AI-analysis records required by enabled features.

SRS-DAT-002 **(Schema inspection and arithmetic tests)** The system shall represent authoritative monetary amounts using integer currency minor units or fixed-precision decimal values.

SRS-DAT-003 **(Database constraint inspection)** Each persisted dispute shall reference its transaction and customer, and each persisted event shall reference its dispute and actor where applicable.

SRS-DAT-004 **(Architecture inspection and Redis-failure test)** Redis shall not be the sole durable copy of a dispute, transaction, audit event or other authoritative record.

SRS-DAT-005 **(Migration test)** The system shall define a versioned database migration path for schema changes.

SRS-DAT-006 **(Inspection)** Retention periods, data residency, deletion requests, encryption-key management and production data classification are TBD and shall be approved before live personal or financial data is processed.

6 Non-Functional Requirements

6.1 Security and Privacy

SRS-NFR-SEC-001 **(Security test suite)** All protected resources shall enforce server-side authentication, ownership and role checks on every request.

SRS-NFR-SEC-002 **(Static analysis and security tests)** The system shall validate and encode untrusted input at appropriate boundaries to prevent injection and cross-site scripting.

SRS-NFR-SEC-003 **(Security test)** The system shall protect state-changing browser requests against cross-site request forgery when the selected authentication mechanism is susceptible to it.

SRS-NFR-SEC-004 **(Upload security test)** Evidence processing shall treat file names, extensions, MIME declarations and file contents as untrusted.

SRS-NFR-SEC-005 **(Repository scan and deployment inspection)** Secrets shall be supplied through the approved deployment secret mechanism and shall not be committed to source control.

SRS-NFR-SEC-006 **(Deployment configuration inspection)** The system shall use transport encryption for production network paths carrying credentials, evidence or protected customer data.

SRS-NFR-SEC-007 **(Threat-model and security-test review)** The system shall document and test controls against broken access control, replay, duplicate submissions, race conditions, sensitive-data leakage and prompt injection.

6.2 Reliability and Integrity

SRS-NFR-REL-001 **(Database failure-injection test)** A failed dispute mutation shall not leave partially committed dispute, event, audit or idempotency records.

SRS-NFR-REL-002 **(Failure-injection test)** A Redis outage shall not cause loss of committed authoritative dispute data.

SRS-NFR-REL-003 **(Failure-injection test)** An AI or worker outage shall not corrupt dispute state or prevent non-AI case operations.

SRS-NFR-REL-004 **(Concurrency test)** The system shall preserve the one-active-dispute-per-transaction invariant under concurrent requests.

6.3 Performance and Scalability

SRS-NFR-PER-001 **(Performance test)** The API shall target a P_{95} response time below 200 ms and a P_{99} response time below 500 ms for non-AI endpoints under a documented benchmark workload.

SRS-NFR-PER-002 **(Benchmark report inspection)** A performance result shall identify hardware, dataset size, concurrency, duration and software configuration.

SRS-NFR-PER-003 **(Integration and performance test)** Long-running AI operations shall execute asynchronously when synchronous execution would violate the configured API latency objective.

SRS-NFR-PER-004 **(Multi-replica integration test)** The backend shall support multiple API replicas without relying on process-local state for authoritative user sessions, dispute state or uniqueness enforcement.

6.4 Maintainability and Portability

SRS-NFR-MNT-001 **(Clean-environment acceptance test)** A fresh checkout shall be startable using the documented local setup procedure and supported container configuration.

SRS-NFR-MNT-002 **(Architecture inspection)** The backend domain layer shall not depend directly on HTTP, Redis or PostgreSQL implementation details.

SRS-NFR-MNT-003 **(CI pipeline test)** Continuous integration shall fail when required formatting, lint, test or production-build checks fail.

SRS-NFR-MNT-004 **(Clean-environment acceptance test)** Kubernetes deployment shall remain optional for local evaluation of the core system.

6.5 Usability and Accessibility

SRS-NFR-USE-001 **(Usability inspection)** Customer and analyst views shall label dispute state and communicate the next permitted action without presenting advisory AI output as authoritative.

SRS-NFR-USE-002 **(Usability test)** The user interface shall make validation failures, stale-update conflicts and unavailable optional AI functions understandable without exposing sensitive implementation details.

SRS-NFR-USE-003 **(Inspection)** Accessibility conformance target and supported browser matrix are TBD and shall be approved before release acceptance.

7 Verification and Acceptance

Each requirement's verification method defines its minimum evidence. Testable functional behaviour shall be covered by automated unit, integration, API or end-to-end tests as appropriate. Security-sensitive and concurrency requirements require negative or race-focused tests, not only successful-path demonstrations.

The system is acceptable for the baseline demonstration when:

1. A fresh checkout can be started using the documented local instructions.
2. Seeded customers can authenticate and view only their own transactions.
3. An eligible transaction can produce one persisted dispute, with a stable reference and visible timeline.
4. Concurrent or repeated submissions cannot produce duplicate active disputes.

5. Invalid transitions, stale updates and unauthorised access are rejected.
6. Evidence and audit events are associated with the correct case and actor.
7. Analysts can review deterministic risk signals without those signals making a final decision.
8. If AI is enabled, invalid output and unavailable services are handled without breaking the core dispute workflow.
9. Automated tests, formatting and lint checks pass in CI.
10. Performance claims, if made, include the required benchmark conditions and results.

8 Traceability

The following map connects requirement groups to the corresponding engineering specification topics. Individual requirement identifiers remain the authoritative references for implementation and verification.

Requirement IDs	**Related engineering-specification topics**
SRS-UI-* and SRS-API-*	Frontend Architecture; API Design; Definition of Done
SRS-FUN-001–006	Authentication and Authorisation; Role-Based Access Control
SRS-FUN-010–013	Scope; Customer Workspace; Dispute Creation Transaction
SRS-FUN-020–034	Dispute State Machine; Duplicate Dispute Prevention; Idempotency; Optimistic Concurrency; Audit Model
SRS-FUN-040–047	Evidence Upload Security; Analyst Workspace; Database Design
SRS-FUN-050–055	Mathematical Risk Engine; Risk Interpretation
SRS-FUN-060–068	AI Architecture; Structured AI Output; Agent Safety Bounds; AI Tool Allowlist
SRS-FUN-070–076	Audit Model; Observability; Metrics; Container Health; Failure Behaviour
SRS-DAT-*	Domain Model; Database Design; Financial Data Representation; Redis Design
SRS-NFR-SEC-*	Security Threat Model; Prompt Injection Defence; Configuration and Secrets
SRS-NFR-REL-*	Duplicate Dispute Prevention; Dispute Creation Transaction; Failure Behaviour
SRS-NFR-PER-*	Latency Objectives; Horizontal Scaling; Performance Testing
SRS-NFR-MNT-*	Proposed Repository Structure; CI/CD; Docker Deployment

9 Open Decisions

The following decisions must be settled by the product or operational owner before they become implementation policy:

- eligibility rules, dispute reasons and any filing time limits;
- the exact active-state set and complete state/role transition matrix;
- customer and analyst authentication mechanism, session policy and administrator provisioning process;
- evidence size and type limits, storage provider, scanning policy, retention and deletion rules;

- idempotency-key format, lifetime, storage and expiry policy;
- risk weights, signal definitions, missing-data policy, reference data, thresholds and currency normalisation;
- AI model selection, data handling, timeout, budget, retention and availability policy;
- rate-limit values and behaviour during Redis failure;
- API schemas, error contract, pagination and versioning policy;
- supported browsers, accessibility target, production hosting, recovery objectives and operational service levels;
- applicable legal, privacy, financial-services and regulatory obligations for any non-demo deployment.